

О языке постановки задач автоматизации программирования

Андрей П. Немытых
Институт программных систем РАН
г. Переславль-Залесский

Совместное рабочее совещание ИПС РАН и МГТУ имени Н.Э. Баумана
по функциональному языку программирования Рефал

1 июля 2026 г., г. Переславль-Залесский

Искусственный «интеллект» превратит человека в обезьяну

“Метафора о «превращении человека обратно в обезьяну» из-за искусственного «интеллекта» отражает реальные риски цифрового слабоумия, атрофии когнитивных навыков и утраты критического мышления.”

(Искусственный «интеллект», июнь, 2026 г.)

Искусственный «интеллект» превратит человека в обезьяну

Глубина разложения человеческого общества продолжает увеличиваться:

XIX-ый век Физический труд человека стал быстро заменяться работой машин. Это привело к потере природного физического здоровья человека.

XX-ый век Развитие химии – в частности, для вынужденного поддержания этого здоровья, является причиной ослабления иммунитета. Более того, иммунитет перестраивается на принятие различных химических веществ, произведенных искусственным путем, не совпадающих с их природными аналогами, если таковые вообще существуют, и отвержение природных защитных веществ и механизмов.

Искусственный «интеллект» превратит человека в обезьяну

Глубина разложения человеческого общества продолжает увеличиваться:

XXI-ый век Мы – современники начала процесса отказа от умственной работы. Замены её тыканьем на кнопки и получением случайной выборки из накопленной громадной базы данных посредством очень быстрой поисковой машины, по недоразумению называемой искусственным «интеллектом».

Искусственный «интеллект» превратит человека в обезьяну

К стыду своему, докладчик будет бросать камни в оба огорода . . .

Задача и подзадачи

Будем различать (начальную) задачу и подзадачи этой задачи.

Любая задача предполагает наличие двух субъектов:

- того, кто ставит (формулирует) задачу;
- и того, кто будет решать задачу;
 - иногда эти субъекты совпадают.
- Вопросы:
 - Может ли «ИИ» сам себе ставить задачи?
 - Может ли программа сама себе ставить задачи?
 - Дана программа P . Может ли копия программы P ставить задачу программе P ?

Любая задача предполагает наличие двух субъектов

Будем различать (начальную) задачу и подзадачи этой задачи.

Того, кто (что) ставит (начальную) задачу, будем называть мета-субъектом по отношению к субъекту, который решает задачу:

- Преподаватель (семинарист) является мета-субъектом по отношению к студенту.
- Человек, который (пока) обычно ставит задачу программе, есть мета-субъект по отношению к программе.
 - Студент, решающий задачу, поставленную преподавателем, посредством некоторой программы, есть мета-объект по отношению к этой программе.

Студент и программа $P(x, y)$

- Как студент ставит задачу вычисления для данной программы $P(x, y)$?

Студент и программа $P(x, y)$

Как студент ставит задачу вычисления для данной программы $P(x, y)$?

- Он указывает точку (x_0, y_0) из множества определения этой программы и просит программу вычислить её значение в этой точке:
 - $P(x_0, y_0)$

Студент и программа $P(x, y)$

Как студент ставит задачу вычисления для данной программы $P(x, y)$?

- Он указывает точку (x_0, y_0) из множества определения этой программы и просит программу вычислить её значение в этой точке:
 - $P(x_0, y_0)$
- Что делает программа – как происходит процесс вычисления?

Студент и программа $P(x, y)$

Как студент ставит задачу вычисления для данной программы $P(x, y)$?

- Он указывает точку (x_0, y_0) из множества определения этой программы и просит программу вычислить её значение в этой точке:
 - $P(x_0, y_0)$
- Что делает программа – как происходит процесс вычисления?
 - Используя текущие значения своих переменных:
 - присваивает их значения и константы другим переменным;
 - вычисляет значения функций (подпрограмм) и стандартных операций в конкретных точках, определяемых текущими значениями переменных;
 - присваивает значения этих функций некоторым переменным;
 - и т.д.
 - Какое очень важное действие здесь пропущено?

Студент и программа $P(x, y)$

Как студент ставит задачу вычисления для данной программы $P(x, y)$?

- Он указывает точку (x_0, y_0) из множества определения этой программы и просит программу вычислить её значение в этой точке: $P(x_0, y_0)$
- Что делает программа – как происходит процесс вычисления?
 - Используя текущие значения своих переменных:
 - присваивает их значения и константы другим переменным;
 - вычисляет значения функций (подпрограмм) и стандартных операций в конкретных точках, определяемых текущими значениями переменных;
 - присваивает значения этих функций некоторым переменным; и т.д. ...
 - Какое очень важное действие здесь пропущено?
 - Начальные значения переменных этими действиями не определяются!

Студент и программа $P(x, y)$

Как студент ставит задачу вычисления для данной программы $P(x, y)$?

- Он указывает точку (x_0, y_0) из множества определения этой программы и просит программу вычислить её значение в этой точке: $P(x_0, y_0)$
- Аргументы входной точки программы определяются студентом – метасубъектом по отношению к программе:
 - либо через командную строку операционной системы;
 - либо руками – посредством редактирования входной точки программы;
 - либо стандартными библиотечными функциями ввода.

Студент и программа $P(x, y)$

Как студент ставит задачу вычисления для данной программы $P(x, y)$?

- Он указывает точку (x_0, y_0) из множества определения этой программы и просит программу вычислить её значение в этой точке: $P(x_0, y_0)$
- Аргументы входной точки программы определяются студентом – метасубъектом по отношению к программе:
 - либо через командную строку операционной системы;
 - либо руками – посредством редактирования входной точки программы;
 - либо стандартными библиотечными функциями ввода.
- **Выразительных языковых средств** в субъектном языке для инициализации входных аргументов программы **не существует**.

Студент и программа $P(x, y)$

Как студент ставит задачу вычисления для данной программы $P(x, y)$?

- Он указывает точку (x_0, y_0) из множества определения этой программы и просит программу вычислить её значение в этой точке: $P(x_0, y_0)$
- Аргументы входной точки программы определяются студентом – метасубъектом по отношению к программе.
- Что представляют собой входные аргументы программы?
 - Студент выбирает конкретную точку из области определения программы, и ставит задачу вычисления для программы на этой входной точке – упорядоченном множестве конкретных констант.
 - Программа ожидает одну конкретную точку (x_0, y_0) из множества входных данных, но не знает какую именно из этих точек студент подаст ей на вход.

Параметры vs. переменные

Квадратное уравнение с параметрами:

$$x^2 + p * x + q = 0$$

, где **x** есть переменная, а **p, q** суть параметры.

- Как решаются уравнения с параметрами?
- В чём разница между переменными и параметрами?

Параметры vs. переменные

В простейшем случае студент ставит задачу:

Дано определение (программа) частичной функции

$$f(\mathbf{x}, \mathbf{y}) : \mathbf{K} \times \mathbf{M} \rightarrow \mathbf{D}$$

, где $\mathbf{D}$ есть множество данных и $\mathbf{K} \subset \mathbf{D}, \mathbf{M} \subset \mathbf{D}$. Входную точку этой программы (т.е. задачу для этой программы) будем обозначать так:

$$\langle f \ \# \mathbf{x}, \# \mathbf{y} \rangle$$

Здесь, $\# \mathbf{x}$ и $\# \mathbf{y}$ суть параметры, множества значений которых суть $\mathbf{K}, \mathbf{M}$, соответственно.

- Начальный вызов функции будем обозначать угловыми скобками, чтобы подчеркнуть, что язык постановки задач является метаязыком относительно всех конкретных языков программирования. (В.Ф. Турчин)

Студент и программа $P(x, y)$

Любое вычисление программы на конкретных входных данных можно понимать как упрощение этой программы:

- замену её на результат вычисления.

Какая программа стоит между студентом и программой $P(x, y)$, которую он запускает?

- На каком языке студент видит результат вычисления?

Студент, интерпретатор-посредник Int_L и программа $P(x, y)$
 Студент ставит задачу интерпретатору, а интерпретатор ставит задачу программе:

Входную точку этой программы (т.е. задачу для Int_L) будем обозначать так:

$$\begin{array}{c} \langle \text{Int}_L \quad \#x, \#y, \text{def}(P(x, y)) \rangle \\ \quad \quad \quad \downarrow \quad \downarrow \\ \quad \quad \quad \bullet \quad \bullet \\ \langle f \quad \bullet, \bullet \rangle \end{array}$$

Здесь f – входная функция программы $P(x, y)$, $\#x$ и $\#y$ суть параметры, значения которых должны быть закодированы, перед тем как будут переданы функции f .

- В.Ф. Турчин назвал такие двумерные схемы постановки задач схемами мета-системных переходов.
- Сдвиг вызова (или выражения) на вторую строку равносильен однострочной схеме, в которой вторая строка закодирована и возвращена обратно – точно вверх. Т.е. вторая строка является аргументом первой строки, который должен подаваться закодированным.

Студент, интерпретатор-посредник Int_L и программа $P(x, y)$

Входную точку этой программы (т.е. задачу для Int_L) будем обозначать так:

$$\langle \text{Int}_L \quad \begin{array}{c} \#x, \#y \\ \downarrow \quad \downarrow \\ \bullet \quad \bullet \end{array} , \text{def}(P(x, y)) \rangle$$

Здесь f – входная функция программы $P(x, y)$, $\#x$ и $\#y$ суть параметры, значения которых должны быть закодированы, перед тем как будут переданы функции f .



$$\langle \text{Int}_L \quad \mu[\langle f \quad \#x, \#y \rangle], \text{def}(P(x, y)) \rangle$$

Здесь μ – функция кодировки, $\text{def}(P(x, y)) := \mu[\text{def}'(P(x, y))]$.

- Int_L – интерпретатор языка L , на котором написана программа $P(x, y)$

Студент, интерпретатор-посредник Int_L и программа $P(x, y)$

Входную точку этой программы (т.е. задачу для Int_L) будем обозначать так:

$\langle \text{Int}_L \quad \#x, \#y \quad , \quad \rangle$

$\langle f \bullet, \bullet \rangle , \text{def}'(P(x, y))$

Здесь f – входная функция программы $P(x, y)$, $\#x$ и $\#y$ суть параметры, значения которых должны быть закодированы, перед тем как будут переданы функции f .



$\langle \text{Int}_L \quad \mu[\langle f \quad \#x, \#y \rangle], \text{def}(P(x, y)) \rangle$

Здесь μ – функция кодировки, $\text{def}(P(x, y)) := \mu[\text{def}'(P(x, y))]$.

- Int_L – интерпретатор языка L , на котором написана программа $P(x, y)$.
- На каком языке выдается результат вычисления этой задачи?

Оптимизатор, интерпретатор Int_L и программа $P(x, y)$

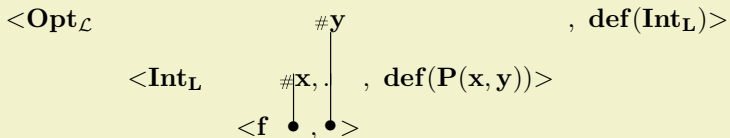
Заменим студента оптимизатором программ:

$$\begin{array}{c} \langle \text{Opt}_L \qquad \qquad \qquad , \text{def}(\text{Int}_L) \rangle \\ \qquad \langle \text{Int}_L \qquad \#x, \#y \quad , \text{def}(P(x, y)) \rangle \\ \qquad \qquad \qquad \downarrow \quad \downarrow \\ \qquad \qquad \langle f \quad \bullet, \bullet \quad \rangle \end{array}$$

- Какая семантика результата вычисления этой задачи?
- Т.е. что является результатом работы оптимизатора Opt_L , преобразующего программы, написанные на языке $\mathcal{L}$?

Оптимизатор, интерпретатор Int_L и программа $P(x, y)$

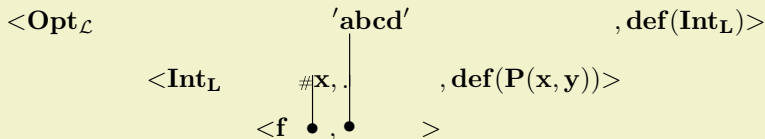
Какую задачу ставит следующее описание?



- Что является результатом работы оптимизатора Opt_L , преобразующего программы, написанные на языке L ?

Оптимизатор, интерпретатор Int_L и программа $P(x, y)$

Частный случай постановки задачи – $\#y_0 = \text{'abcd'}$.



- Что является результатом работы оптимизатора Opt_L , преобразующего программы, написанные на языке L ?

Оптимизатор, интерпретатор Int_L и программа $P(x, y)$

Возможный результат преобразований при $\#y_0 = \text{'abcd'}$.

```
// Программа P(x,y) на языке программирования L.  
// Входная точка:  
bool f(string x, string y) {  
    if(x == y) then { return true; };  
    return false;  
}
```

```
// Результат преобразований,  
// данный на языке программирования  $\mathcal{L}$ .  
// Входная точка bool main(string x):  
main('abcd') = True;  
main(x) = False;
```


Оптимизатор, интерпретатор Int_L и программа $P(x, y)$

Возможный результат преобразований при $\#y = \#char$.

```
// Программа P(x,y) на языке программирования L.  
// Входная точка:  
bool f(string x, char y) {  
    if(x == (string)y) then { return true; };  
    return false;  
}
```

```
// Результат преобразований,  
// данный на языке программирования  $\mathcal{L}$ .  
// Входная точка bool main(string x):  
main('a') = True;  
main('b') = True;  
main(x) = False;
```

Почему получается?

Мы научились компилировать посредством интерпретатора и оптимизатора программ?

По данным программе $P(x, y)$ на языке L и интерпретатору $\mathbf{Int}_L$ этого языка, написанному на языке $\mathcal{L}$, научились компилировать $P(x, y)$ из языка L в язык $\mathcal{L}$, посредством оптимизатора программ, написанных на языке $\mathcal{L}$?

- Что такое интерпретатор?
- Что такое компилятор?
- Почему один лучше или хуже другого?

Компиляция

Компилятор Comp_L языка L в язык M

Данна программа $P(x, y)^L$ на языке L .

$$\text{Comp}_L(\underline{P(x, y)^L}) = P'(x, y)^M$$

Программы $P(x, y)^L$ и $P'(x, y)^M$ определяют (реализуют) одну и ту же частичную функцию.

- Программа $P'(x, y)^M$ представляет собой «исполняемый модуль».
- Исполнение:
 - Shell> $P' \ x_0 \ y_0$

Интерпретация

Интерпретатор Int_L языка L

Данна программа $P(x, y)^L$ на языке L .

$\langle \text{Int}_L \quad \begin{array}{c} \#x, \#y \\ \downarrow \quad \downarrow \\ \bullet \quad \bullet \end{array} , \text{def}(P(x, y)^L) \rangle$
 $\langle f \bullet, \bullet \rangle$

- Интерпретатор Int_L представляет собой «исполняемый модуль».
- Исполнение:
 - Shell> $\text{Int}_L \ P \ x_0 \ y_0$
- Создадим командный модуль $P.\text{bat}$ состоящий из команды:
 $\text{Int}_L \ P \ \%x \%y$, тогда:
 - Shell> $P \ x_0 \ y_0$

Интерпретация

Интерпретатор Int_L языка L

Данна программа $P(x, y)^L$ на языке L .

$\langle \text{Int}_L \quad \begin{array}{c} \#x, \#y \\ \downarrow \quad \downarrow \\ \bullet \quad \bullet \end{array} , \text{def}(P(x, y)^L) \rangle$
 $\langle f \bullet, \bullet \rangle$

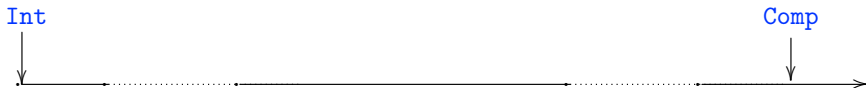
Интерпретатор Int_L можно спрятать в «исполняемый модуль».

```
int main(int int, char * argv[]) {  
    char * x = argv[1];  
    char * y = argv[2];  
    return Int_L_PxyL(x, y);  
}  
int Int_L_PxyL(char * x, char * y) {  
    //.....  
}
```

Почему получается?

Интерпретатор Int_L есть операционная семантика языка L .

- Почему это не фокус:
 - Не существует чёткой грани между интерпретацией и компиляцией.



Как часто будет получаться содержательная компиляция?

Интерпретатор Int_L есть операционная семантика языка L .

- Почему это не фокус:
 - Не существует чёткой грани между интерпретацией и компиляцией.
 - Будет ли всегда получаться содержательная компиляция?



Как часто будет получаться содержательная компиляция?

Интерпретатор Int_L есть операционная семантика языка L .

- Почему это не фокус:
 - Будет ли всегда получаться содержательная компиляция?
 - Если язык программирования L алгоритмически полный, тогда невозможно надеяться на положительный ответ.



Как часто будет получаться содержательная компиляция?

Интерпретатор Int_L есть операционная семантика языка L .

- Почему это не фокус:
 - Будет ли всегда получаться содержательная компиляция?
 - Если язык программирования L не является алгоритмически полным, тогда получение ответа на поставленный вопрос является интересной задачей.



Оптимизатор, интерпретатор Int_L и подмножество множества данных

Ограничение области определения программ – $\#y_0 = \text{'abcd'}$.

$$\langle \text{Opt}_L \quad \langle \text{Prop} \langle {}^t\text{Int}_L \#time, \quad \begin{array}{c} \text{'abcd'} \\ | \\ \bullet \end{array}, \quad \begin{array}{c} \#x, \\ | \\ \bullet \end{array}, \quad \#prog_2 \rangle \rangle, \text{def}({}^t\text{Int}_L) \rangle$$

- ${}^t\text{Int}_L$ – интерпретатор частичного выполнения программ, написанных на языке L :
 - $\#time$ – дискретное время исполнения (число шагов интерпретатора);
 - если за время $\#time$ результат программы ещё не вычислен, тогда ${}^t\text{Int}_L$ возвращает своё текущее состояние,
 - иначе ${}^t\text{Int}_L$ возвращает результат вычисления программы.
- Prop – предикат свойства надежности состояния ${}^t\text{Int}_L$.

Оптимизатор, интерпретатор Int_L и подмножество множества данных

Ограничение области определения программ – $\#y_0 = \text{'abcd'}$.

$$\langle \text{Opt}_L \quad \langle \text{Prop} \langle {}^t\text{Int}_L \#time, \quad \begin{array}{c} \text{'abcd'} \\ | \\ \bullet \end{array}, \quad \begin{array}{c} \text{'abcd'} \\ | \\ \bullet \end{array}, \#prog_2 \rangle \rangle, \text{def}({}^t\text{Int}_L) \rangle$$

$\begin{array}{c} \#x, \\ | \\ \bullet \end{array} \quad \begin{array}{c} \text{'abcd'} \\ | \\ \bullet \end{array}$
 $\langle f \bullet, \bullet \rangle$

- Здесь ${}^t\text{Int}_L$ – интерпретатор частичного выполнения программ:
 - $\#time$ – число шагов интерпретатора;
 - если за время $\#time$ результат программы ещё не вычислен, тогда ${}^t\text{Int}_L$ возвращает своё текущее состояние,
 - иначе ${}^t\text{Int}_L$ возвращает результат программы;
- **Prop** – предикат свойства надежности состояний ${}^t\text{Int}_L$.
- Opt_L есть оптимизатор программ, написанных на языке $\mathcal{L}$.

ЛИТЕРАТУРА

1. V.F. Turchin, A.P. Nemytykh. To Friends (Friends.to). E-Mail, The City University of New York, 04.04.1994, New York. (URL http://refal.botik.ru/library/Valetin-F-Turchin_Friends-To_04-04-1994.pdf)
2. V.F. Turchin, A.P. Nemytykh. Metavariables: their Implementation and Use in Program Transformation. The City College of the City University of New York, Report N. TR 2095-012, 1995, 34 p.
3. V.F. Turchin, A.P. Nemytykh, V.A. Pinchuk. A Self-Applicable Supercompiler. In: Partial Evaluation, LNCS vol. 1110, 1996, pp. 322-337.
4. Yo. Futamura. Partial Evaluation of Computation Process – An Approach to a Compiler-Compiler. Higher-Order and Symbolic Computation 12, 381–391 (1999).
5. N.D. Jones. Computability and Complexity From a Programming Perspective. The MIT Press, Massachusetts London, 1997.